

Team Weekly Journal - Week 4

Project: Intelligent Security Log Summarization and Threat Timeline Generation

Course: CSC 482 Capstone Project 2

Team: Project Team 2

Week: Week 4, June 22-June 28, 2026

Focus: LLM Integration and Structured Summarization

GitHub Repository: <https://github.com/Dredman72/security-log-threat-timeline>

Milestones Achieved

Done	Milestone	Brief Description
[x]	Refined OpenAI summarization prompt	Derrick improved the OpenAI prompt so the model is instructed to return concise, structured JSON report output for security logs.
[x]	Improved fallback behavior	Derrick improved the application behavior when model output is incomplete, invalid, or missing expected fields.
[x]	Confirmed report sections	The team confirmed the main report sections: executive summary, risk level, attack type, affected assets, indicators of compromise, key findings, threat timeline, evidence, and recommended actions.
[x]	Connected parser output direction to LLM input	Zion prepared AI-ready parser output so normalized backend events can support Derrick's OpenAI summarization workflow.
[]	Frontend report-section design	Oriah still needs to provide Week 4 evidence for the summary/report section design in the Flask/HTML direction.

Subtasks Completed

Member	Subtask	Brief Description
Derrick	Refined OpenAI prompt	Updated prompt expectations so OpenAI returns structured report sections that match the project milestone checklist.
Derrick	Added report normalization	Improved app behavior so missing fields are filled with safe defaults such as Unknown, empty lists, or Unknown event.
Derrick	Improved JSON cleanup	Added logic to handle common JSON formatting problems, including model output wrapped in code blocks or extra text around JSON.
Derrick	Improved upload handling	Clarified supported file types and added clearer behavior for raw .evtx files and oversized uploads.
Zion	Parser-to-LLM input preparation	Prepared normalized parser output in an AI-ready structure that can support OpenAI summarization and report generation.
Oriah	Frontend report planning	Pending final Week 4 evidence. Oriah needs to provide a report-section layout, screenshot, mockup, or frontend update showing executive summary, risk, findings, timeline, and recommendations.

Testing of Subtasks With Demo Screenshot Evidence

Test 1: OpenAI Prompt Structure Review

Purpose: Confirm the prompt asks for the correct report sections.

Procedure: Derrick reviewed the prompt in `app.py` and confirmed it lists the required output fields.

Expected Result: The model should return JSON fields for executive summary, risk level, attack type, affected assets, indicators of compromise, key findings, timeline, evidence, and recommended actions.

Result: Passed. The prompt clearly defines the required report structure.

Screenshot Evidence: Insert screenshot of `app.py` prompt section or generated report output.

Test 2: Sample Log Analysis

Purpose: Confirm the app can generate a structured report from suspicious log activity.

Procedure: Derrick pasted sample SSH and firewall logs into the Flask app and clicked Analyze Logs.

Expected Result: The app should return a readable security report with summary, risk level, attack type, IOCs, findings, timeline events, evidence, and recommended actions.

Result: Passed. The app produced a structured report that was reviewed as demo-ready.

Screenshot Evidence: Insert screenshot showing sample logs and generated report output.

Test 3: Threat Timeline Review

Purpose: Confirm the timeline includes evidence and chronological event details.

Procedure: The team reviewed the generated timeline section from the sample SSH/firewall log test.

Expected Result: Each timeline event should show timestamp, source, severity, event title, details, and evidence.

Result: Passed. Timeline events included evidence from the original logs.

Screenshot Evidence: Insert screenshot of the timeline section.

Test 4: Fallback Behavior Review

Purpose: Confirm the app does not crash if model output is incomplete or imperfect.

Procedure: Derrick added normalization logic so missing report fields are filled safely and invalid JSON output falls back to a review message.

Expected Result: The app should not fail silently or crash when the model output needs cleanup.

Result: Passed by code review and local function checks.

Screenshot Evidence: Insert screenshot of successful output or test confirmation.

Test 5: Upload Handling Review

Purpose: Confirm the app gives clear upload feedback.

Procedure: Derrick reviewed supported upload extensions and added clear messages for unsupported files, raw `.evtx` files, and files larger than the configured limit.

Expected Result: The app should give a clear message instead of crashing or failing silently.

Result: Passed by code review.

Screenshot Evidence: Insert screenshot of upload form or error message if tested.

Test 6: AI-Ready Parser Input Review

Purpose: Confirm Zion's Week 4 parser output can support the LLM workflow.

Procedure: The team reviewed the AI-ready parser input structure against the report fields needed by Derrick's OpenAI prompt.

Expected Result: Parser output should contain enough normalized event detail to support summary, risk level, attack type, affected assets, IOCs, key findings, timeline evidence, and recommended actions.

Result: Passed. Zion's Week 4 parser-to-LLM input task is complete for the current project stage.

Screenshot Evidence: Insert screenshot of the AI-ready parser input file or related documentation.

Pending Test: Frontend Report Section Design

Purpose: Confirm Oriah's Week 4 report-section layout is ready for the shared Flask/HTML prototype.

Procedure: Oriah should provide a screenshot, mockup, or frontend update showing the report sections in browser/report form.

Expected Result: The report display should clearly show executive summary, risk, findings, timeline, and recommendations.

Result: Pending.

Lessons Learned

- The LLM prompt must be specific because the frontend depends on consistent output sections.
- Fallback behavior is important because model output can sometimes be incomplete or formatted unexpectedly.
- The team needs a clear shared structure between parser output, OpenAI input, and frontend display.
- Raw .evt.x files need special parsing support, so the current prototype should use exported .xml, .txt, .log, .csv, or .json files.
- GitHub documentation should be updated every week so the repository reflects current progress.

Contribution of Each Team Member

Team Member	Role	Week 4 Contribution
Derrick Redman	Team Leader + AI/LLM Specialist	Refined the OpenAI prompt, improved structured JSON output handling, added fallback behavior, updated documentation, and continued coordinating backend/frontend alignment.
Zion Moore	Backend & Log Processing Lead	Prepared AI-ready parser input and continued connecting normalized backend output to the LLM/report workflow.
Oriah Molton-Bowman	Frontend & Visualization Lead	Completed Week 3 frontend planning and still needs to submit Week 4 evidence for the report-section design task.

Progress Compared to Plan

The team has progressed according to the project plan. Derrick's Week 4 LLM tasks are complete:

- Refine OpenAI summarization prompt.
- Improve fallback handling for model output.

Zion's Week 4 parser-to-LLM input preparation is also complete. The remaining Week 4 work is Oriah's report-section design based on the finalized structured output.

Plan Adjustment

No major adjustment is required. The team will continue using Flask/HTML as the shared prototype framework.

The only adjustment is coordination-based: Oriah should use Zion's sample normalized parser output and Derrick's confirmed report labels to finish the Week 4 frontend/report section design.

Team Sign-Off

Team Member	Role	Signature	Date
Derrick Redman	Team Leader + AI/LLM Specialist		
Zion Moore	Backend & Log Processing Lead		
Oriah Molton-Bowman	Frontend & Visualization Lead		